

JobReady AI

PRODUCT DISCOVERY | Phase 1 — Product Requirements Document

Turning a life-ending moment into a life-changing phase — through right guidance, awareness and information.

Product	JobReady AI
Version	v1.0 — MVP + Future Enhancements
Phase	Phase 1 — Product Discovery & Definition
Owner	Aditya Nagaria — Product & Program Manager
Audience	Self reference GitHub portfolio PM interview presentations
Status	Draft — In Progress

1. EXECUTIVE SUMMARY

JobReady AI is a GenAI-powered job search companion built for mid-career professionals facing involuntary job loss. The product addresses a specific, under-served problem: capable professionals with 8–15 years of experience applying frantically, without feedback, without direction and without support — turning a temporary career disruption into a prolonged life crisis.

This PRD defines the requirements for three core MVP features: the **JD Analyser**, the **Application Tracker**, and the **Mock Interview Engine**. Together these features attack the three biggest failure points in Rahul's journey — the targeting gap, the visibility gap and the preparation gap. Each feature is defined with user stories, MVP requirements, future enhancements and acceptance criteria.

2. FOUNDATION — PROBLEM, PERSONA & JOURNEY

Problem Statement	Mid-career professionals facing involuntary job loss are trapped in a system they cannot see, understand or navigate alone — turning a temporary career disruption into a prolonged life crisis. Full document: JobReadyAI_ProblemStatement.pdf
User Persona	Rahul — 37 years old, Senior Software Engineer, 11 years experience, laid off 6 weeks ago. Breadwinner. Young child. Financially committed. Applying frantically without feedback or direction. Full document: JobReadyAI_UserPersona.pdf
Journey Entry Point	JobReady AI enters in Week 4 — after 3 weeks of unguided, unstructured failure. Rahul is at his lowest. The product's first job is to make him feel understood before it makes him feel equipped. Full document: JobReadyAI_UserJourneyMap.pdf

3. PRODUCT GOALS & SUCCESS METRICS

GOAL	USER OUTCOME	SUCCESS METRIC
Reduce application-to-callback ratio	Rahul gets a callback within 2 weeks of using JD Analyser	Callback rate improves from <5% to >20% for targeted applications
Break the paralysis loop	Daily Momentum actions completed 5 out of 7 days	Daily active usage in Weeks 4–6 of job search

Improve interview readiness	Rahul scores 70%+ in Mock Interview Engine within 3 sessions	Self-reported confidence score increases by 2 points after 3 sessions
Restore structured visibility	All applications logged and tracked in Application Tracker	Zero applications sent without being logged — 100% pipeline visibility
Emotional outcome	User reports feeling 'in control' after Week 1 of product use	NPS > 60 from beta users in same persona bracket

JD Analyser

JOURNEY MOMENT

Week 4 — Rahul's first interaction with JobReady AI. He has been applying blindly for 3 weeks. This is the first time the system gives him a diagnosis instead of a rejection.

PROBLEM SOLVED

Rahul cannot see why his applications fail. He does not know if the gap is skills, resume language, seniority mismatch or role targeting. The JD Analyser makes the invisible visible.

User Story: As Rahul, I want to paste a job description and get a personalised fit score against my profile so that I understand exactly where my gaps are and what actions to take — instead of guessing.

MVP REQUIREMENTS

- Accept job description as free text input (paste or upload)
- Accept user profile as structured input — skills, experience, last role, target role
- Generate a fit score (0–100) with a confidence indicator
- Produce a gap analysis: matched skills, missing skills, partially matched skills
- Return 3–5 specific, actionable improvement recommendations
- Flag role-level mismatch — e.g. JD expects 15 years, profile shows 11
- Identify ATS keywords in JD not present in user profile
- Output readable in under 60 seconds

FUTURE ENHANCEMENTS

- Batch analysis — analyse 5 JDs simultaneously and rank by fit
- Resume rewrite suggestions — specific line edits to improve keyword match
- Market comparison — how does this JD compare to similar roles in the market
- Trend tracking — are the skills this JD requires growing or declining in demand
- Save and compare — track fit score improvement over time as profile is updated
- Integration with Application Tracker — auto-log analysed JDs

ACCEPTANCE CRITERIA

AC-01	Given a valid JD and user profile, the system returns a fit score within 60 seconds
AC-02	Fit score must differentiate between exact match, partial match and missing skills — not a single number alone
AC-03	Gap analysis must surface at least 3 specific missing skills or mismatches
AC-04	Recommendations must be actionable and specific — not generic (e.g. 'Add Agile to resume' not 'Improve your resume')
AC-05	ATS keyword extraction must identify at least 5 high-frequency terms from the JD
AC-06	If role-level mismatch exists (years of experience, seniority), system must surface this explicitly
AC-07	Output must be readable without technical knowledge — plain language, no jargon

CONSTRAINTS & DEPENDENCIES

- LLM dependency — output quality depends on Gemini / OpenAI model version in use
- No real-time job market data in MVP — gap analysis based on JD text only
- User profile is self-reported — accuracy of analysis depends on quality of input
- Google Colab environment — no persistent database in MVP, session-based only
- Token limits — very long JDs may need truncation before processing

Application Tracker

JOURNEY MOMENT

Week 5 — Rahul is now applying with more structure. But he has no record of what he sent, where it stands or which roles are going cold. The Tracker gives him a pipeline view for the first time.

PROBLEM SOLVED

Rahul's job search is invisible to him. He cannot see progress, cannot identify which applications are stalling, and cannot learn from the pattern. The Tracker turns chaos into a managed pipeline.

User Story: As Rahul, I want to log every job application I send and track its status through the pipeline so that I always know where each opportunity stands and can take timely follow-up actions.

MVP REQUIREMENTS

- Log an application: company, role title, JD link, date applied, source (portal/referral)
- Track application status: Applied / Viewed / Shortlisted / Interview Scheduled / Rejected / Offer
- Add notes per application — interview date, interviewer name, prep notes
- View full pipeline in a table — sortable by date, status, company
- Flag stale applications — no status change in 7+ days
- Summary dashboard: total applied, in progress, rejected, offer rate
- Export pipeline as CSV or PDF report

FUTURE ENHANCEMENTS

- AI insights — pattern analysis: which role types are getting callbacks, which are not
- Follow-up nudges — automated reminders to follow up on stale applications
- JD Analyser integration — fit score auto-populated at time of logging
- Recruiter contact log — track who you spoke to and when
- Offer comparison tool — compare multiple offers on compensation, role, growth
- Visual pipeline — Kanban-style drag-and-drop board view

ACCEPTANCE CRITERIA

- | | |
|-------|--|
| AC-01 | User can log a new application in under 2 minutes with all required fields |
| AC-02 | Status updates are reflected in the dashboard immediately upon save |
| AC-03 | Stale applications (7+ days, no status change) are flagged visually — colour or icon |
| AC-04 | Dashboard summary metrics are accurate — total counts match logged entries |
| AC-05 | CSV export contains all logged fields and is openable in standard spreadsheet tools |
| AC-06 | Pipeline table is sortable by at least 3 columns: date, status, company name |
| AC-07 | System handles 50+ logged applications without performance degradation |

CONSTRAINTS & DEPENDENCIES

- MVP is session-based in Google Colab — data does not persist between sessions without manual CSV export
- No email integration in MVP — status updates are manually entered by user
- No real-time job portal sync — application data is self-reported
- AI insights feature deferred to Phase 2 — MVP is data capture and visibility only
- Mobile optimisation not in scope for MVP — desktop / Colab interface only

Mock Interview Engine

JOURNEY MOMENT

Week 5–6 — Rahul has callbacks coming. He has not done a structured interview in 3+ years. He avoids practice because it surfaces gaps he is not ready to face. The Mock Engine meets him where he is.

PROBLEM SOLVED

Rahul's interview preparation is generic and unstructured. He reads tips, watches videos, but does not practice. When the actual interview arrives, he is underprepared for the specific role and level. The Mock Engine fixes this.

User Story: As Rahul, I want to practice answering role-specific interview questions with AI feedback so that I can identify my weak areas, improve my answers before the real interview and walk in with genuine confidence.

MVP REQUIREMENTS

- Accept target role and JD as input to generate role-specific questions
- Generate 10 questions per session: 4 technical, 3 behavioural, 2 situational, 1 culture fit
- Present one question at a time — user types or speaks their answer
- Evaluate answer against: relevance, completeness, structure (STAR format for behavioural), clarity
- Provide specific feedback per answer — what worked, what was missing, what to add
- Generate a session score (0–100) with breakdown by question type
- Suggest an improved answer for questions scored below 60
- Allow session replay — review all Q&A; with feedback after session ends

FUTURE ENHANCEMENTS

- Voice input and transcription — speak answers, not just type them
- Session history — track score improvement across multiple sessions
- Difficulty progression — questions get harder as scores improve
- Panel simulation — multi-persona interview (technical + HR + hiring manager)
- Salary negotiation module — practice compensation conversations
- Company-specific preparation — questions tailored to target company's known interview style

ACCEPTANCE CRITERIA

AC-01	Questions generated are specific to the JD provided — not generic question lists
AC-02	Question set must include all 4 types: technical, behavioural, situational, culture fit
AC-03	Feedback on each answer must reference the specific answer given — not generic advice
AC-04	STAR format evaluation is applied only to behavioural questions — not technical
AC-05	Session score is reproducible — same answer to same question scores within ± 5 points
AC-06	Improved answer suggestions are provided for all answers scoring below 60
AC-07	Session replay shows original question, user answer, feedback and suggested answer
AC-08	System must not penalise a correct technical answer for lack of STAR structure

CONSTRAINTS & DEPENDENCIES

- Answer evaluation quality depends on LLM — subjective answers may receive variable scoring
- No audio processing in MVP — voice input is a future enhancement

- Session data not persisted between Colab sessions in MVP — export required to save progress
- LLM may not have current knowledge of company-specific interview formats
- Evaluation of highly technical answers may be limited by LLM's domain depth

4. OUT OF SCOPE — MVP BOUNDARY

The following features and capabilities are explicitly out of scope for the MVP. They are acknowledged, valued and deferred — not dismissed. Scope discipline is product discipline.

Daily Momentum Generator	Identified as a high-value feature but deferred. Will be included in Phase 2 Build Sprint after core engine is validated.
Interview Question Generator	Standalone feature deferred. Interview question generation is partially covered within the Mock Interview Engine in MVP.
Real-time job market data	No live API integration with job portals (Naukri, LinkedIn) in MVP. All data is user-input or JD-text-based.
Mobile application	Web or mobile app not in scope. MVP runs on Google Colab — desktop environment only.
User authentication	No login, user accounts or cloud persistence in MVP. Data is session-based, exportable to CSV/PDF.
Payment / monetisation	No pricing model implemented in MVP. Product is built to demonstrate capability and validate hypothesis first.
Multi-language support	English only in MVP. Hindi and regional language support is a post-validation enhancement.
Resume builder	Resume rewrite is a future enhancement of JD Analyser. Not a standalone MVP feature.

5. ASSUMPTIONS & CONSTRAINTS

ASSUMPTIONS	CONSTRAINTS
• The target user (Rahul) is tech-comfortable enough to use a Colab-based tool • Gemini API or OpenAI API will be available and accessible during build and demo • User will self-report profile data honestly — analysis quality depends on input quality • English-language JDs are the primary use case for MVP • The product will be demonstrated in a Google Colab environment for portfolio purposes • GitHub will serve as the public-facing portfolio and documentation repository	• Google Colab is the coding and demo environment — no persistent backend in MVP • LLM output quality is non-deterministic — same input may produce varied output • No real-time data access — market insights are LLM knowledge-based, not live • ReportLab is the PDF generation library — formatting complexity is bounded by its capabilities • Solo builder — scope is intentionally constrained to what one person can build and document • 2-month bench timeline — MVP must be completable within this window

6. OPEN QUESTIONS

OPEN QUESTION	CONTEXT & OPTIONS	DECISION BY
How is fit score calculated?	Is it a weighted average of matched skills? Does seniority match carry more weight than skill match? Needs definition before build.	Phase 2
What is the profile input format?	Does the user paste their resume, fill a structured form, or both? Structured form is more reliable but higher friction at onboarding.	Phase 2

How is answer quality evaluated?	What rubric does the Mock Engine use for technical vs behavioural answers? LLM prompt engineering will determine this — needs iteration.	Build Sprint 1
How does the product handle emotion?	If a user expresses frustration or despair in their input, how does the product respond? Needs a content and tone policy.	Phase 2
What is the monetisation model?	Free tool for portfolio? Freemium with premium features? This affects onboarding design and feature gating decisions.	Post-MVP
Who validates the product?	Is there a beta user group from the target persona? Even 3–5 real users would strengthen the hypothesis validation.	Post-MVP

This PRD is a living document.

It will be updated as assumptions are validated, open questions are answered and the build sprint surfaces new decisions.

Every requirement here traces back to a moment in Rahul's journey. That is the test of a good PRD.